ماژول‌ها، کتابخانه‌ها و مدیریت فایل‌ها و پوشه‌ها

هنگام ساخت یک نرم‌افزار، لازم نیست همه قابلیت‌ها را از ابتدا بنویسیم. بسیاری از عملیات موردنیاز ما قبلاً پیاده‌سازی شده‌اند و می‌توانیم از آن‌ها استفاده کنیم. همچنین، برنامه‌ها معمولاً باید اطلاعات را در فایل‌ها ذخیره کنند یا فایل‌ها و پوشه‌های موجود در سیستم را مدیریت کنند.

در این مبحث با دو موضوع کاربردی آشنا می‌شویم: استفاده از ماژول‌ها و کتابخانه‌های پایتون و کار با فایل‌ها و پوشه‌ها.

۱. ماژول و کتابخانه چیست؟

در پایتون، می‌توانیم کدهای مرتبط را در فایل‌هایی جداگانه قرار دهیم و در برنامه‌های دیگر از آن‌ها استفاده کنیم. به چنین فایل‌هایی که کد پایتون در آن‌ها قرار دارد، ماژول (Module) می‌گوییم.

برای مثال، فرض کنید تابعی برای محاسبه مالیات نوشته‌ایم. می‌توانیم آن را در یک فایل جداگانه نگهداری کنیم و در برنامه‌های دیگر به کار ببریم.

کتابخانه (Library) مجموعه‌ای از کدهای آماده است که برای انجام کارهای مشخص در اختیار برنامه‌نویس قرار می‌گیرد. کتابخانه‌ها ممکن است شامل چندین ماژول باشند.

پایتون امکانات آماده بسیاری دارد؛ علاوه بر آن، برنامه‌نویسان و شرکت‌ها کتابخانه‌های دیگری نیز توسعه داده‌اند که می‌توانیم در پروژه‌های خود استفاده کنیم.

۲. استفاده از ماژول‌های آماده

پایتون همراه با مجموعه‌ای از ماژول‌های استاندارد عرضه می‌شود. این ماژول‌ها برای انجام بسیاری از کارهای رایج کاربرد دارند و معمولاً نیازی به نصب جداگانه ندارند.

برای استفاده از یک ماژول، از دستور `import` استفاده می‌کنیم.

برای مثال، ماژول `math` امکاناتی برای انجام محاسبات ریاضی در اختیار ما قرار می‌دهد:

```
import math

result = math.sqrt(25)

print(result)
```

خروجی:

```
5.0
```

در این مثال، تابع `sqrt()` از ماژول `math` برای محاسبه جذر عدد استفاده شده است.

با دستور `math import` ماژول را وارد برنامه کرده‌ایم و با نوشتن `math.sqrt()` مشخص کرده‌ایم که تابع موردنظر متعلق به این ماژول است.

وارد کردن بخش مشخصی از یک ماژول

گاهی فقط به یک تابع یا مقدار از یک ماژول نیاز داریم. در این حالت می‌توانیم آن را مستقیماً وارد کنیم:

```
from math import pi

radius = 5
area = pi * radius ** 2

print(area)
```

در این مثال، مقدار `pi` را از ماژول `math` وارد کرده‌ایم و از آن برای محاسبه مساحت دایره استفاده کرده‌ایم.

بهتر است کدی بنویسیم که مشخص باشد هر تابع یا مقدار از کجا آمده است. به همین دلیل، معمولاً از وارد کردن تمام اعضای یک ماژول با دستورهایی مانند `from math import *` خودداری می‌کنیم.

۳. ساخت ماژول اختصاصی

خودمان نیز می‌توانیم ماژول بسازیم. برای این کار کافی است کد موردنظر را در یک فایل پایتون ذخیره کنیم.

فرض کنید دو فایل در یک پوشه داریم:

```
project/
  main.py
  calculations.py
```

در فایل `calculations.py`، تابعی برای محاسبه مجموع دو عدد می‌نویسیم:

```
def add(a, b):
    return a + b
```

سپس در فایل `main.py`، از آن استفاده می‌کنیم:

```
from calculations import add

result = add(10, 20)

print(result)
```

خروجی:

```
30
```

در این مثال، تابع `add()` را فقط یک بار نوشته‌ایم و می‌توانیم در قسمت‌های مختلف برنامه از آن استفاده کنیم.

این روش برای سازمان‌دهی برنامه‌های بزرگ‌تر مفید است. برای مثال، می‌توانیم توابع مربوط به محاسبات، مدیریت کاربران و پردازش داده‌ها را در ماژول‌های جداگانه قرار دهیم.

در این مثال فرض کرده‌ایم هر دو فایل در یک پوشه قرار دارند. ساختار پروژه و نحوه اجرای برنامه باید طوری باشد که پایتون بتواند ماژول موردنظر را پیدا کند.

۴. کتابخانه‌های خارجی

علاوه بر ماژول‌های استاندارد، کتابخانه‌های زیادی وجود دارند که توسط افراد و سازمان‌های دیگر توسعه داده شده‌اند.

برای مثال:

- `requests` برای ارسال درخواست به سرویس‌های وب.
- `pandas` برای پردازش و تحلیل داده‌ها.
- `matplotlib` برای رسم نمودار.

برخلاف ماژول‌های استاندارد، این کتابخانه‌ها معمولاً باید پیش از استفاده نصب شوند.

برای نمونه، نصب کتابخانه `requests` از طریق ابزار مدیریت بسته‌های پایتون معمولاً با دستور زیر انجام می‌شود:

```
python -m pip install requests
```

این دستور در ترمینال سیستم اجرا می‌شود. اگر از یک محیط توسعه یا ویرایشگر کد استفاده می‌کنید، ممکن است امکان نصب کتابخانه‌ها را از طریق بخش مدیریت بسته‌های همان محیط نیز داشته باشید.

پس از نصب، می‌توانیم کتابخانه را در برنامه وارد کنیم:

```
import requests
```

در ادامه، برای استفاده از قابلیت‌های آن، باید با توابع و شیوه کار کتابخانه آشنا شویم.

نکته: برای شروع یادگیری پایتون، نیازی نیست کتابخانه‌های زیادی نصب کنیم. بهتر است ابتدا امکانات استاندارد پایتون را بشناسیم و زمانی به سراغ کتابخانه‌های خارجی برویم که مسئله به آن‌ها نیاز داشته باشد.

۵. چرا برنامه‌ها با فایل‌ها کار می‌کنند؟

تا اینجا بیشتر مثال‌های ما با داده‌هایی کار می‌کردند که هنگام اجرای برنامه در متغیرها نگهداری می‌شدند.

اما معمولاً با پایان اجرای برنامه، مقادیر ذخیره‌شده در متغیرها از بین می‌روند. اگر بخواهیم داده‌ها را برای استفاده در اجرای بعدی حفظ کنیم، می‌توانیم آن‌ها را در یک فایل ذخیره کنیم.

برای مثال، ممکن است یک برنامه نیاز داشته باشد:

- یادداشت‌های کاربر را ذخیره کند.
- اطلاعات محصولات را از یک فایل بخواند.
- گزارش اجرای برنامه را در فایل ثبت کند.
- فهرستی از کارها را نگهداری کند.

پایتون امکانات ساده‌ای برای خواندن و نوشتن فایل‌ها در اختیار ما قرار می‌دهد.

۶. نوشتن در فایل

برای نوشتن اطلاعات در یک فایل متنی می‌توانیم از تابع `open()` استفاده کنیم.

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Hello, Python!\n")
    file.write("This is my first file.")
```

این کد یک فایل متنی به نام `notes.txt` ایجاد می‌کند یا اگر فایل از قبل وجود داشته باشد، محتوای قبلی آن را جایگزین می‌کند.

در این مثال:

- `notes.txt` نام فایل است.
- حالت `"w"` برای نوشتن استفاده می‌شود.
- `encoding="utf-8"` امکان ذخیره متن با نویسه‌هایی مانند حروف فارسی را فراهم می‌کند.
- `write()` برای نوشتن متن در فایل استفاده می‌شود.
- `\n` به معنای رفتن به خط بعد است.
- `with` کمک می‌کند فایل پس از پایان کار به‌درستی بسته شود.

استفاده از `with` روش پیشنهادی برای کار با فایل‌هاست؛ زیرا نیازی نیست فایل را به‌صورت دستی ببندیم.

نکته مهم: حالت `"w"` محتوای قبلی فایل را پاک می‌کند. بنابراین، هنگام کار با فایل‌های موجود باید در انتخاب حالت نوشتن دقت کنیم.

۷. خواندن از فایل

برای خواندن محتوای یک فایل، می‌توانیم از حالت `"r"` استفاده کنیم:

```
with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()

print(content)
```

خروجی:

```
Hello, Python!
This is my first file.
```

در این مثال، متد `read()` تمام محتوای فایل را می‌خواند و در متغیر `content` قرار می‌دهد.

اگر فایل موردنظر وجود نداشته باشد، اجرای این کد خطا ایجاد می‌کند. در برنامه‌های واقعی باید درباره فایل‌های ناموجود یا خطاهای خواندن تصمیم مناسبی بگیریم.

خواندن خط‌به‌خط

گاهی فایل طولانی است و ترجیح می‌دهیم خط‌ها را یکی‌یکی پردازش کنیم:

```
with open("notes.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

در این مثال، حلقه `for` هر بار یک خط از فایل را دریافت می‌کند. متد `strip()` فاصله‌ها و نویسه‌های اضافی ابتدا و انتهای خط، از جمله نویسه پایان خط، را حذف می‌کند.

این روش برای پردازش فایل‌هایی که چندین خط دارند، کاربردی است.

۸. اضافه‌کردن اطلاعات به انتهای فایل

گاهی نمی‌خواهیم اطلاعات قبلی را حذف کنیم، بلکه قصد داریم متن جدیدی به انتهای فایل اضافه کنیم.

برای این کار از حالت `"a"` استفاده می‌کنیم:

```
with open("notes.txt", "a", encoding="utf-8") as file:
    file.write("\nThis line was added later.")
```

اگر فایل از قبل وجود داشته باشد، متن جدید به انتهای آن اضافه می‌شود. اگر وجود نداشته باشد، معمولاً فایل ساخته می‌شود.

تفاوت سه حالت پرکاربرد فایل را در نظر داشته باشید:

حالت	کاربرد
`"r"`	موجود فایل خواندن
`"w"`	قبلی محتوای کردن‌جایگزین و نوشتن
`"a"`	فایل انتهای به اطلاعات کردن‌اضافه

انتخاب حالت مناسب اهمیت دارد؛ زیرا ممکن است با انتخاب اشتباه، اطلاعات قبلی فایل از بین برود.

۹. فایل‌ها و پوشه‌ها با `pathlib`

برای مدیریت مسیر فایل‌ها و پوشه‌ها، پایتون ماژولی به نام `pathlib` دارد. این ماژول امکاناتی فراهم می‌کند که کار با مسیرها را ساده‌تر و مستقل از سیستم‌عامل می‌سازد.

برای مثال:

```
from pathlib import Path

file_path = Path("notes.txt")

print(file_path.exists())
print(file_path.is_file())
```

این کد بررسی می‌کند که آیا مسیر `notes.txt` وجود دارد و آیا به یک فایل اشاره می‌کند یا خیر.

متدهای مهم `Path` عبارت‌اند از:

- `exists()` بررسی می‌کند که مسیر وجود دارد یا خیر.
- `is_file()` بررسی می‌کند که مسیر یک فایل است یا خیر.

- `is_dir()` بررسی می‌کند که مسیر یک پوشه است یا خیر.
- `mkdir()` برای ایجاد پوشه استفاده می‌شود.
- `iterdir()` برای پیمایش محتویات یک پوشه به کار می‌رود.

ایجاد پوشه و ذخیره فایل در آن

فرض کنید می‌خواهیم فایل‌های گزارش را در پوشه‌ای به نام `reports` نگهداری کنیم:

```
from pathlib import Path

reports_dir = Path("reports")
reports_dir.mkdir(exist_ok=True)

report_path = reports_dir / "report.txt"

report_path.write_text(
    "Application started successfully.",
    encoding="utf-8"
)
```

در این مثال:

1. مسیر پوشه `reports` تعریف می‌شود.
2. با `mkdir()` پوشه ایجاد می‌شود؛ اگر از قبل وجود داشته باشد، به‌کمک `exist_ok=True` خطایی ایجاد نمی‌شود.
3. با استفاده از عملگر `/`، مسیر فایل `report.txt` درون پوشه ساخته می‌شود.
4. متد `write_text()` متن را در فایل می‌نویسد.

مانند حالت `"w"` در `open()`، متد `write_text()` نیز محتوای قبلی فایل را جایگزین می‌کند.

استفاده از `pathlib` به ما کمک می‌کند مسیرها را بدون وابستگی به شکل نوشتن مسیرها در لینوکس یا ویندوز مدیریت کنیم.

۱۰. مسیر نسبی و مسیر مطلق

یکی از نکات مهم در کار با فایل‌ها، شناخت مسیر فایل است.

مسیر نسبی (Relative Path) مسیر را نسبت به پوشه کاری فعلی برنامه مشخص می‌کند.

برای مثال:

```
from pathlib import Path

file_path = Path("data") / "notes.txt"
```

این مسیر به فایلی به نام `notes.txt` در پوشه `data` اشاره می‌کند؛ اما محل دقیق این پوشه به پوشه کاری فعلی اجرای برنامه بستگی دارد.

مسیر مطلق (Absolute Path) محل فایل را از نقطه مشخصی در سیستم فایل تعیین می‌کند؛ برای مثال، در لینوکس ممکن است مسیری مانند `/home/user/project/notes.txt` باشد.

بهتر است از نوشتن مسیرهای مطلق وابسته به رایانه شخصی خود در برنامه‌هایی که قرار است دیگران نیز اجرا کنند، خودداری کنیم.

برای بررسی پوشه کاری فعلی می‌توانیم از این دستور استفاده کنیم:

```
from pathlib import Path

print(Path.cwd())
```

این دستور مسیر پوشه کاری فعلی را نمایش می‌دهد.

توجه کنید که پوشه کاری فعلی لزوماً همان پوشه‌ای نیست که فایل پایتون در آن قرار دارد. این موضوع ممکن است با توجه به محیط برنامه‌نویسی و روش اجرای کد متفاوت باشد.

۱۱. مشاهده محتویات یک پوشه

گاهی لازم است فهرستی از فایل‌ها و پوشه‌های موجود در یک مسیر تهیه کنیم.

```
from pathlib import Path

folder = Path("reports")

if folder.exists() and folder.is_dir():
    for item in folder.iterdir():
        print(item.name)
else:
    print("Folder not found.")
```

در این مثال، ابتدا بررسی می‌کنیم که مسیر موردنظر وجود دارد و یک پوشه است. سپس با `iterdir()` محتویات آن را پیمایش می‌کنیم و نام هر مورد را نمایش می‌دهیم.

اگر بخواهیم فقط فایل‌های متنی را پیدا کنیم، می‌توانیم از `glob()` استفاده کنیم:

```
from pathlib import Path

folder = Path("reports")

for file_path in folder.glob("*.txt"):
    print(file_path.name)
```

عبارت `"*.txt"` به فایل‌هایی اشاره می‌کند که پسوند `.txt` دارند.

این قابلیت برای ساخت ابزارهایی که فایل‌ها را جست‌وجو یا دسته‌بندی می‌کنند، مفید است.

۱۲. ترکیب ماژول‌ها و کار با فایل‌ها

اکنون می‌توانیم این مفاهیم را در یک مثال کوچک ترکیب کنیم.

فرض کنید می‌خواهیم نام چند کار را در فایل متنی ذخیره کنیم و سپس آن‌ها را بخوانیم.

```
from pathlib import Path

data_dir = Path("data")
data_dir.mkdir(exist_ok=True)

tasks_file = data_dir / "tasks.txt"

tasks = [
    "Learn Python",
    "Practice functions",
    "Build a small project"
]

with tasks_file.open("w", encoding="utf-8") as file:
    for task in tasks:
        file.write(task + "\n")

print("Saved tasks:")
```

```
with tasks_file.open("r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

در این مثال:

- از ماژول `pathlib` برای ساخت مسیر و مدیریت پوشه استفاده می‌کنیم.
- از یک لیست برای نگهداری کارها کمک می‌گیریم.
- با حلقه `for`، نام کارها را در فایل می‌نویسیم.
- سپس فایل را می‌خوانیم و محتویات آن را نمایش می‌دهیم.

در نتیجه، اطلاعات فقط در حافظه متغیرها باقی نمی‌مانند؛ بلکه در یک فایل نیز ذخیره می‌شوند و می‌توانیم در اجرای بعدی برنامه از آن‌ها استفاده کنیم.

۱۳. نکات مهم هنگام کار با فایل‌ها

هنگام کار با فایل‌ها به چند نکته توجه کنید:

فایل ممکن است وجود نداشته باشد. پیش از خواندن یک فایل، بررسی کنید که آیا وجود دارد یا برای نبودنش راه‌حل مناسبی در نظر بگیرید.

نوشتن ممکن است اطلاعات قبلی را حذف کند. حالت `"w"` و متد `write_text()` محتوای قبلی فایل را جایگزین می‌کنند.

از کدگذاری مناسب استفاده کنید. برای فایل‌های متنی، به‌ویژه زمانی که حروف فارسی در میان هستند، تعیین `encoding="utf-8"` انتخاب مناسبی است.

مسیرها به محیط اجرا وابسته‌اند. برنامه ممکن است در رایانه شما کار کند، اما در محیط دیگری نتواند فایل را پیدا کند؛ زیرا پوشه کاری یا ساختار فایل‌ها متفاوت است.

فایل‌های مهم را با احتیاط تغییر دهید. حذف یا جایگزینی اطلاعات ممکن است قابل بازگشت نباشد. هنگام آزمایش، از فایل‌های نمونه استفاده کنید.

جمع‌بندی

ماژول‌ها و کتابخانه‌ها امکان استفاده مجدد از کدهای آماده را فراهم می‌کنند و کمک می‌کنند برنامه‌ها را به بخش‌های منظم‌تری تقسیم کنیم.

از طرف دیگر، با استفاده از امکاناتی مانند `open()` و `pathlib` می‌توانیم فایل‌ها را بخوانیم و بنویسیم، پوشه بسازیم، مسیرها را مدیریت کنیم و اطلاعات را برای استفاده‌های بعدی ذخیره کنیم.

این مهارت‌ها پایه‌ای برای ساخت برنامه‌های کاربردی‌تر هستند؛ برنامه‌هایی که نه‌تنها محاسبه انجام می‌دهند، بلکه با داده‌های ذخیره‌شده و فایل‌های سیستم نیز کار می‌کنند.

تمرین

برنامه‌ای بنویسید که یک فهرست از کارهای روزانه را در فایل ذخیره کند و سپس آن را بخواند و نمایش دهد.

برنامه باید:

1. پوشه‌ای به نام `data` ایجاد کند، اگر از قبل وجود ندارد.
2. چند کار را در یک لیست نگهداری کند.
3. کارها را در فایل `tasks.txt` داخل پوشه `data` ذخیره کند.
4. فایل را دوباره بخواند و همه کارها را نمایش دهد.
5. از `pathlib` برای مدیریت مسیر فایل و از `with` برای کار با فایل استفاده کند.

تمرین تکمیلی: برنامه را طوری تغییر دهید که کارهای جدید به انتهای فایل اضافه شوند، بدون آنکه کارهای قبلی حذف شوند.